```
      assert_match(/krauss/i, entry.artist)
    end
  end

  def test_title_playlist
    @playlist_builder.include_title("midnight")
    refute_empty(@playlist_builder.playlist, "Playlist shouldn't be empty")
    @playlist_builder.playlist.each do |entry|
      assert_match(/midnight/i, entry.title)
    end
  end

  # ...
end
```

produces:

```
Run options: --seed 40135
# Running:

...
Finished in 0.000466s, 6437.7683 runs/s, 98712.4468 assertions/s.
3 runs, 46 assertions, 0 failures, 0 errors, 0 skips
```

Now, the setup method is creating the @database as a Minitest::Mock and then setting the expectation that the test will call connect and disconnect on the object (which is done behind the scenes by the PlaylistBuilder). The second argument to each method is a value returned when the mocked method is called. At the end of each test, in the teardown method we verify the mock object, which raises a failure if both expectations are not met.

Minitest mock objects can get more complicated. A mock object can take an optional third argument that is an array of arguments, and an optional block argument. If those arguments are used, then the mock object only accepts the method call if the arguments match. If not, it raises a MockExpectationError when called with arguments that don't match. If you want to call the mock object multiple times, you need to have multiple expect calls, which are used in the order defined.

It's common to want to override one method on an existing object rather than create an entire mock object (this is not necessarily recommended, but it is common). In minitest, you can do this with the stub method, which is added to Object, so it's available to all objects.

The first argument to stub is the name of the method you want to intercept, as a symbol. The second argument is the value you want returned, or you can pass a block argument. The return value of the stub is either:

- The value returned by the block if there is a block.
- The result of second_arg.call if the second argument responds to call, usually meaning that it is a Proc or lambda.
- The second argument itself if neither of the first two options are true.

So, we could re-write the setup of that test using stub as follows:

```
def setup
  @database = DBI.new("DBI:mysql:playlists")
  @database.stub(:connect, true)
  @database.stub(:disconnect, true)
  @playlist_builder = PlaylistBuilder.new(@database)
end
```